Technical Product Documentation

Enterprise Incident Intelligence Platform

Purpose: Sample technical documentation for semantic embedding, ChromaDB indexing, and retrieval-augmented generation exercises.

Document Area	Coverage
Product	Features, users, workflows
Architecture	API, agent, database, integrations
Installation	Python environment and dependencies
Configuration	Environment variables and service settings
Operations	Incident workflow and observability
Troubleshooting	Common failures and corrective actions

1. Product Overview

Purpose

The Enterprise Incident Intelligence Platform receives incident information from monitoring and service-management systems, stores normalized incident records, and applies AI-assisted analysis to help support teams investigate production issues.

Key Features

The platform supports incident ingestion, persistent incident storage, severity and context analysis, investigation workflows, retrieval of historical information, integration adapters, and API-based access for operational clients.

Primary Users

Typical users include production support engineers, application support teams, site reliability engineers, incident managers, and technical leads who require consistent incident information and faster investigation.

2. Solution Architecture

Architecture Summary

The solution uses a layered architecture. A FastAPI service exposes HTTP endpoints. Service and repository layers isolate business logic and persistence. PostgreSQL stores structured incident data. Agent components coordinate analysis and tool calls. Integration components communicate with monitoring and service-management platforms.

Core Components

API Layer: request validation and REST endpoints. Service Layer: orchestration and business rules. Repository Layer: database access. Agent Layer: investigation and reasoning workflows. Integration Layer: connectors for external systems. Configuration Layer: environment-driven settings and secrets.

Typical Data Flow

A monitoring event enters through an integration endpoint. The application validates and normalizes the payload, creates or updates an incident record, invokes analysis services when required, persists results, and exposes the enriched incident through the API. External ticketing integration can associate the internal incident with a service-management record.

3. Installation Procedure

Prerequisites

Use a supported Python environment, PostgreSQL database access, and network access to any configured external endpoints. Create a dedicated virtual environment so project dependencies remain isolated.

Create Virtual Environment

From the project directory create and activate a virtual environment before installing packages.

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
```

Install Dependencies

Install the application dependencies from the project requirements file. Typical dependencies include FastAPI, Uvicorn, SQLAlchemy, a PostgreSQL driver, python-dotenv, LangChain libraries, and HTTP client packages.

```
python -m pip install -r requirements.txt
```

4. Configuration Guidelines

Environment File

Store runtime configuration in a .env file located at the project root. Do not commit production credentials to source control. Inject sensitive values using the approved secrets-management process for the target environment.

```
DB_HOST=127.0.0.1
DB_PORT=5432
OPENAI_BASE_URL=https://example.invalid/v1
OPENAI_EMBEDDING_MODEL=
CHROMA_DB_FOLDER=chroma_technical_db
```

Database Configuration

Define database host, port, database name, user, and password separately. Construct the SQLAlchemy database URL from these settings. Validate connectivity before starting workloads that require persistence.

AI Configuration

Keep chat-model and embedding-model configuration separate. A chat model generates natural-language responses, while an embedding model converts document text into numeric vectors for semantic search. The embedding model used during retrieval must be compatible with the vectors created during ingestion.

5. Document Embedding and ChromaDB

Ingestion Pipeline

Load technical_doc.pdf, extract page text, split the text into manageable overlapping chunks, generate an embedding vector for each chunk, and write the vectors plus source metadata into a persistent ChromaDB collection.

```
PDF -> Text Extraction -> Chunking -> Embeddings -> ChromaDB
Query -> Query Embedding -> Similarity Search -> Relevant Chunks
```

Chunking Strategy

A practical starting configuration is approximately 1000 characters per chunk with 200 characters of overlap. Overlap preserves limited context across chunk boundaries. The exact values should be tuned for document structure and the selected embedding model.

Metadata

Store metadata such as source file, page number, chunk number, and collection name. Metadata makes retrieved chunks traceable to the original document.

Semantic Retrieval

For a search query, generate a query embedding using the same compatible embedding model, compare that vector with stored vectors, and return the most similar chunks. Downstream RAG applications can pass retrieved chunks to a chat model as grounded context.

6. Operational Workflow

Incident Creation

An incoming event is validated, assigned an internal incident identifier, persisted, and returned through the service API. Integration-specific identifiers can be stored to maintain traceability between systems.

AI-Assisted Investigation

The investigation layer can combine incident attributes, historical context, approved knowledge sources, and external tool results. System integrations should remain observable so failures can be separated from model or application failures.

Logging

Log application startup, database connectivity failures, integration requests, tool execution, and unexpected exceptions. Avoid recording API keys, passwords, access tokens, or other secrets in logs.

7. Troubleshooting Guide

Database Connection Failure

Verify database host, port, credentials, database availability, firewall rules, and the generated connection URL. Test basic connectivity before debugging higher application layers.

Embedding Request Returns 403

A 403 response commonly indicates that the provided identity or key is not authorized for the requested embedding resource. Verify the configured endpoint, embedding model entitlement, and credential scope. A chat-model credential does not automatically prove embedding access.

Embedding Model Not Found

Verify the exact embedding model identifier supported by the configured OpenAI-compatible endpoint. Do not substitute a chat model name where an embedding model is required.

ChromaDB Returns No Results

Confirm that ingestion completed successfully, the expected collection contains records, the same persistent directory is opened during retrieval, and the retrieval code uses an embedding configuration compatible with ingestion.

PDF Produces No Text

The PDF may contain scanned images rather than extractable text. Inspect the document and use OCR before embedding when the PDF loader returns empty or unusable page content.

API Fails to Start

Review the first traceback rather than only the final error line. Validate Python environment activation, package imports, .env loading, database initialization, and occupied network ports.

8. Validation Checklist

Ingestion Validation

Confirm the PDF is found, pages are loaded, chunks are created, a test embedding succeeds, ChromaDB files are created, vector records are stored, and a semantic search returns relevant chunks.

RAG Readiness

The knowledge repository is ready for downstream RAG when document vectors are persisted, source metadata is retained, retrieval can reopen the collection, and representative questions return relevant source chunks.

Security Validation

Keep credentials outside source files, avoid printing sensitive configuration, apply least-privilege access to databases and external APIs, and sanitize logs before sharing troubleshooting evidence.

End of Technical Documentation